RAG Pipeline - Advanced Indexing: RAPTOR

1. Definition

RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval) is an advanced indexing technique that builds a **hierarchical tree of summaries** from a collection of documents.

Unlike "flat" indexing which only stores small, uniform chunks, RAPTOR creates multiple layers of abstraction. This allows the retrieval system to answer both highly specific, "low-level" queries (by matching raw text chunks) and broad, thematic "high-level" queries (by matching the abstract summaries).

2. Core Process (Indexing)

The RAPTOR indexing process is a recursive, bottom-up tree-building algorithm. It transforms raw text into a structure where the "leaves" are details and the "roots" are high-level concepts.

Step 1: Preparation (The Leaves)

- **Splitting:** The raw documents are split into small, contiguous chunks (e.g., 100-400 tokens).
- **Leaf Nodes:** These raw chunks form **Level 0** (the bottom layer) of the tree.

Step 2: The Recursive Loop

This is the heart of RAPTOR. The system performs the following cycle repeatedly until it reaches the top of the tree:

1. **Embedding & Reduction:**
    - The current layer's nodes (initially the raw chunks) are embedded into vectors.
    - **UMAP** is applied to reduce the dimensionality of these vectors, making the next step faster and more accurate.
2. **Clustering (GMM):**
    - The reduced vectors are fed into a **Gaussian Mixture Model (GMM)** to find semantic clusters.
    - **Soft Clustering:** Crucially, a node can belong to *multiple* clusters if it is relevant to different themes (e.g., a chunk about "Pricing" might belong to both the "Sales" cluster and the "Finance" cluster).
    - **Optimization:** The **Bayesian Information Criterion (BIC)** is used to automatically determine the optimal number of clusters, so you don't have to guess.

3. **Summarization:**
    - For *each* cluster found, an LLM reads all the text within that cluster.
    - The LLM generates a concise **summary** of that cluster.
4. **Rise to Next Level:**
    - These new **summaries** become the nodes for **Level 1**.
    - The cycle repeats: Level 1 summaries are embedded $\rightarrow$ clustered $\rightarrow$ summarized to create **Level 2**.

Step 3: Termination

The recursion stops when:

- The clustering algorithm can no longer find distinct groups (meaning the data is unified).
- A single **Root Node** (a summary of the entire corpus) is reached.
- A predefined maximum tree depth is hit.

3. Core Process (Retrieval)

Retrieval uses a "collapsed tree" approach:

1. The user's query is embedded.
2. The vector is searched against the *entire* vector store (containing leaf chunks, Level 1 summaries, Level 2 summaries, etc.).
3. **Result:**
    - **Specific queries (Low Level Query)** match leaf nodes (e.g., "What is the create_sql_query_chain?").
    - **Thematic queries (High Level/Broad Query)** match summary nodes (e.g., "What are the main principles of query construction?").
    - This allows the LLM to "zoom in" or "zoom out" dynamically based on the user's intent.

4. Key Methodologies & Terms

Methodology	Role in RAPTOR
GMM (Gaussian Mixture Model)	Probabilistic clustering model. Allows chunks to belong to multiple clusters (soft clustering).
BIC (Bayesian Information Criterion)	Used to mathematically calculate the "optimal" number of clusters for GMM, avoiding manual guessing.
UMAP	Dimensionality reduction technique. Reduces complex embeddings (e.g., 1536d) to simpler ones (e.g., 10d) to make clustering faster and more accurate.
Local & Global Clustering	A strategy to perform clustering at different "resolutions"—finding huge global themes first, then finding tiny local themes within them.

5. Key LangChain Tools

RAPTOR is an advanced *pattern* rather than a single, off-the-shelf retriever in the core library. It is implemented by combining several core LangChain components.

Component	Reason for Use
Text Splitters	To create the initial "leaf nodes" (Level 0) of the tree (Source 1.6).
Embedding Model	To create vector representations for all nodes (both chunks and summaries) at every level (Source 1.2).
Clustering Algorithm	(e.g., sklearn.mixture.GaussianMixture) To group similar nodes together so they can be summarized (Source 1.6).
LLM Chain	A chain dedicated to taking a list of documents (the chunks in a cluster) and generating an abstractive summary for them (Source 1.2).
VectorStore	A single vector store (e.g., Chroma, FAISS) is used to store the embeddings for *all* nodes from *all* levels of the tree.
VectorStoreRetriever	A standard retriever is used to search the entire, multi-level index. The retrieval logic itself doesn't need to be hierarchical, because the *data* is.
LangChain Cookbook	The LangChain team has provided a cookbook on GitHub that shows a complete, from-scratch implementation of this pattern (Source 1.5).

6. When to use RAPTOR?

Use RAPTOR when your business case fits these criteria:

- **Data:** Large corpus of documents where "global knowledge" is as important as specific facts.
- **Queries:** Users ask high-level questions ("What is the main strategy of this company?") as often as specific ones ("What was the Q3 revenue?").
- **Constraint:** You need higher accuracy for complex queries and can afford the upfront cost of generating summaries.